SANTACRUZ POS

Descripcion tecnica para el registro de software

Fecha de elaboracion: 5 de octubre de 2026. Documento basado en el codigo disponible en este repositorio; no constituye una certificacion de cumplimiento tributario ni una declaracion de titularidad.

1. Identificacion y objeto

SANTACRUZ POS es un sistema de informacion para la gestion de puntos de venta, con orientacion a restaurantes y operaciones comerciales. Integra pedidos, mesas, cocina, domicilios, facturacion, caja, cartera, productos, inventario, compras, reportes y administracion de accesos.

El nombre del sistema, los paquetes y las rutas de despliegue utilizan la denominacion POS.

El objetivo funcional es relacionar la operacion de venta con sus documentos, pagos y efectos sobre existencias, conservando registros estructurados para consultas administrativas y trazabilidad. La disponibilidad de funciones depende del rol, permisos, parametrizacion y servicios externos configurados.

2. Arquitectura

La solucion utiliza una arquitectura cliente-servidor de tres capas:

1. Presentacion: aplicacion web React, construida con Vite y navegacion con React Router. Presenta formularios, tablas, tableros operativos, ventanas de operacion y reportes.
2. Aplicacion: API HTTP desarrollada con Node.js y Express. Sus rutas, bajo `/api`, gestionan las operaciones del negocio, autenticacion, permisos y acceso a datos.
3. Persistencia: base de datos relacional PostgreSQL, accedida mediante Prisma ORM. Los esquemas y las migraciones describen entidades, campos, relaciones y cambios del modelo.

Flujo general: navegador -> API Express -> Prisma -> PostgreSQL. Para la emision electronica, la API se comunica ademas con Factus. La impresion y exportacion se resuelven mediante componentes del cliente y servicios especificos del servidor.

El codigo separa pantallas en `client/src/pages`, componentes reutilizables en `client/src/components`, rutas del servidor en `server/src/routes` y el modelo relacional en `server/prisma`. La capa de plataforma registra restaurantes; el administrador de tenants crea y reutiliza clientes Prisma asociados a la URL de

base de datos de cada restaurante. Esta implementacion permite resolver conexiones por restaurante, sin que su existencia implique por si sola aislamiento certificado entre empresas: la topologia y el control de acceso efectivos deben comprobarse en cada instalacion.

El cliente comunica el contexto seleccionado mediante `x-tenant-id` y envia el token de sesion en la cabecera `Authorization: Bearer`. En desarrollo, Vite puede reenviar las peticiones de API; en despliegue web, Nginx publica el cliente y encamina el acceso al servidor. La envoltura de escritorio separa el proceso de API y el servidor estatico del cliente.

3. Tecnologia utilizada

Componente	Tecnologia declarada en el proyecto	Funcion
Interfaz	React 18.3, React DOM	Construccion de la interfaz
Navegacion	React Router DOM 6.28	Rutas y vistas
Compilacion web	Vite 6, plugin React	Desarrollo y empaquetado
Iconografia	Lucide React	Iconos de la interfaz
Mapas	Leaflet 1.9	Vistas cartograficas de seguimiento
Exportacion	ExcelJS 4.4	Generacion de archivos de calculo
Servidor	Node.js, Express 4.21	API y reglas de negocio
Persistencia	PostgreSQL, Prisma 6.2, pg 8.13	Datos relacionales y conexiones
Autenticacion	JSON Web Token, bcryptjs	Tokens de acceso y tratamiento de contrasenas
Configuracion	dotenv	Variables de entorno
Despliegue	Docker, Docker Compose, Nginx	Empaquetado y publicacion
Escritorio	Electron 33.2	Ventana de escritorio y procesos locales

Las versiones corresponden a rangos declarados en los manifiestos; la version exacta instalada se determina con los archivos de bloqueo y el entorno de despliegue. Las bibliotecas y servicios de terceros no se presentan como desarrollos propios.

4. Modulos funcionales

Area	Alcance identificado
Ventas y mesas	Consulta de mesas y pedidos, seleccion de productos, vinculacion del personal y gestion del flujo de venta
Cocina	Seguimiento de preparaciones y coordinacion con los pedidos
Domicilios	Registro y seguimiento de pedidos para entrega, repartidor y vistas de ubicacion

Area	Alcance identificado
Facturacion	Consulta y generacion de facturas; pantalla denominada Factura de venta; documentos internos y electronicos segun tipo configurado
Caja	Aperturas, movimientos, cierre y asignacion de cajas
Cartera	Cuentas por cobrar, pagos parciales y consulta de obligaciones
Productos	Catalogo, categorias, precios, impuestos y unidades de medida
Kits	Composicion de productos mediante componentes e insumos
Inventario	Existencias, bodegas y movimientos relacionados con entradas, salidas y ajustes
Compras	Registro de proveedores y compras con detalles
Documentos tributarios	Resoluciones, notas credito, notas debito y retenciones, con alcance electronico especifico descrito en la seccion 7
Parametros	Empresa, companias, centros de operaciones, cajas, tipos de documento y catalogos auxiliares
Administracion	Usuarios, roles, permisos y auditoria
Reportes	Ventas, facturas, consumo de insumos y operacion de restaurante

Algunos catalogos auxiliares se implementan mediante pantallas CRUD compartidas. Su existencia no equivale a un sistema contable integral ni a soporte automatico de todas las obligaciones fiscales.

5. Base de datos y modelo de informacion

El esquema principal incluye entidades para usuarios, roles, permisos, auditoria, clientes, mesas, categorias, productos, componentes de kits, inventario, bodegas, movimientos, pedidos, preparaciones de cocina, facturas, detalles, abonos, caja, documentos y compras.

Las relaciones permiten vincular encabezados con detalles y conectar productos, clientes y documentos. Por ejemplo, las facturas conservan sus lineas de detalle; los kits relacionan un producto con sus componentes; los roles se relacionan con permisos y usuarios mediante entidades de asociacion. Existen modelos para companias y centros de operacion, asi como un esquema de plataforma para restaurantes.

El modelo incorpora identificadores UUID en entidades como Producto e Inventario, e identificadores enteros autoincrementales en entidades como InventarioItem y KitComponente. Las claves foraneas vinculan productos, categorias, impuestos, unidades, insumos y documentos. La receta exige una combinacion unica de producto e insumo para cada componente. Algunos importes y cantidades se modelan como Float; no se afirma que todo el modelo utilice aritmetica decimal exacta.

Las facturas contienen campos asociados al numero asignado por Factus, codigo CUFE, enlace del documento y estado de integracion. Las migraciones versionadas documentan la evolucion de la estructura. Las rutas de facturacion usan transacciones de Prisma para agrupar cambios locales; el envio a un servicio externo es una operacion adicional y no una transaccion distribuida con la DIAN.

La persistencia operativa se realiza en PostgreSQL. El navegador tambien conserva informacion de sesion y preferencias de interfaz en almacenamiento local; este almacenamiento no reemplaza la base relacional ni una estrategia de copias de seguridad.

6. Inventarios, productos y kits

El modelo distingue existencias de productos por bodega (Inventario) e insumos de recetas (InventarioItem). Los movimientos pueden referenciar producto, bodega o insumo, tipo, cantidad, costo, fecha y documento de origen. Los kits relacionan un producto con insumos y las cantidades necesarias para producir una unidad.

La logica compartida de kits agrupa los insumos requeridos multiplicando cantidad de producto por cantidad de cada componente, comprueba stock disponible y registra salidas o reversiones. En el flujo de restaurante, el descuento ocurre al tomar el pedido para reflejar lo enviado a preparacion; facturar ese pedido cierra y cobra sin descontar nuevamente. La factura directa, sin pedido previo de mesa, incorpora su propio descuento de insumos. Las modificaciones o cancelaciones que invocan la logica de reversion generan entradas de insumos.

Los efectos sobre existencias, devoluciones y ajustes deben interpretarse segun el tipo de operacion y su configuracion. No se afirma que cualquier documento descuente automaticamente inventario ni que la simple emision de una nota devuelva siempre stock.

7. Integracion con facturacion electronica

El servidor incluye un cliente de integracion con Factus, servicio externo utilizado para el flujo de facturacion electronica. La autenticacion utiliza OAuth 2.0 con credenciales configuradas en el servidor y almacenamiento temporal del token en memoria hasta su vencimiento.

El adaptador transforma clientes, tipos de identificacion, medios de pago, productos, cantidades, precios e impuestos al formato esperado por el proveedor. Para facturas utiliza la operacion `/v2/bills/validate`; para notas credito utiliza `/v2/credit-notes/validate`. Los rangos de numeracion se suministran mediante configuracion.

El valor predeterminado de la URL del proveedor corresponde a un ambiente sandbox. Por ello, la presencia de esta integracion no acredita que una instalacion este habilitada en produccion, que sus credenciales sean validas o que un documento haya sido aceptado por la DIAN.

La ruta de facturacion guarda la referencia y la informacion electronica devuelta por el proveedor. Ante un fallo del envio puede conservar la factura local con estado ERROR, para permitir revision y reintento mediante una ruta de reenvio. Esto separa el registro de la operacion comercial de la disponibilidad del servicio externo; no elimina la necesidad de verificar aceptacion ni los riesgos de duplicacion al repetir una operacion.

Se distingue entre documentos internos y electronicos mediante parametrizacion. Las pantallas de notas debito y retenciones no deben describirse como emisiones electronicas ante la DIAN sin comprobar un adaptador y una respuesta de aceptacion para esos documentos. La disponibilidad del proveedor, la habilitacion del emisor y la configuracion tributaria son requisitos externos.

8. Reportes

La navegacion contempla ventas totales; ventas netas por dia, semana y mes; ventas por producto por periodo; ventas por categoria, mesera, mesa y forma de pago; detalle de facturas; gastos de insumos por producto; consumo de insumos; pedidos por mesa y preparacion de cocina.

Las consultas permiten analizar la informacion registrada. Los resultados dependen de los filtros, la fecha comercial, permisos y calidad de los datos. La capacidad de exportar o imprimir debe confirmarse en cada vista; la presencia de ExcelJS no implica que todas las pantallas exporten.

9. Seguridad y trazabilidad

La API cuenta con middleware de autenticacion y controles por permiso en las rutas protegidas. La interfaz limita las opciones visibles segun permisos del usuario. Los modulos de administracion gestionan roles y accesos, y el modelo incluye registros de auditoria.

Las medidas identificadas no equivalen a una auditoria de seguridad independiente. El responsable de la instalacion debe administrar secretos, certificados TLS, cuentas, acceso a PostgreSQL, actualizaciones, respaldos y proteccion de datos personales. No deben incluirse credenciales ni informacion privada en los anexos de registro.

10. Instalacion y operacion

La aplicacion web se construye con Vite. El servidor requiere Node.js y acceso a PostgreSQL. El repositorio contiene configuracion para despliegue en contenedores y publicacion mediante Nginx.

La variante de escritorio utiliza Electron con una ventana BrowserWindow, aislamiento de contexto y sin integracion directa de Node.js en la pagina. Inicia el servidor de API y sirve el cliente compilado en procesos o servicios locales separados, con puertos predeterminados 4010 y 4011. Los documentos internos de impresion pueden abrirse en ventanas de la aplicacion; los enlaces externos se delegan al navegador del sistema. No se ha verificado un instalador distribuible ni operacion sin PostgreSQL, y esta arquitectura no debe describirse como venta offline con sincronizacion automatica.

En instalaciones con datos existentes, la inicializacion y el sembrado no deben ejecutarse como un paso rutinario: las instrucciones del proyecto advierten sobre operaciones capaces de reemplazar datos o afectar esquemas. Las copias de seguridad, restauracion y continuidad operativa son responsabilidades del despliegue y deben documentarse por separado.

11. Elementos para documentar la obra

Para identificar la implementacion presentada pueden aportarse el codigo fuente propio, la organizacion de modulos, esquemas y migraciones, reglas de operacion, componentes de interfaz, adaptador de facturacion y el manual con capturas. Estos elementos describen la obra; por si solos no prueban titularidad, originalidad juridica ni fecha de creacion.

Antes de radicar, completar y validar:

- Nombre legal del autor o autores y del titular de derechos.
- Version o identificador de entrega y referencia de revision del repositorio.
- Fechas reales de creacion y publicacion, con sus soportes.
- Relacion de aportes propios, dependencias de terceros y licencias aplicables.
- Correspondencia entre POS y la marca comercial visible en las capturas.
- Alcance de la cesion o contratos, cuando existan, y requisitos de la autoridad de registro.

12. Referencias verificables del repositorio

- Presentacion y despliegue.
- Dependencias del cliente.
- Dependencias del servidor.
- Navegacion y permisos de interfaz.
- Modelo principal.
- Modelo de plataforma.
- Registro de rutas API.
- Adaptador Factus.
- Cliente HTTP y contexto de restaurante.
- Administrador de conexiones por restaurante.
- Logica de inventario de recetas.
- Flujo de facturacion e integracion.
- Aplicacion de escritorio.

El manual de usuario y sus anexos visuales se entregan como documentos complementarios. Las capturas deben identificar si corresponden a una instalacion operativa o a una demostracion con datos ficticios.
